

LC 2115: Find All Possible Recipes from Given Supplies

Authored by: Arpit Raj, LNMIIT Jaipur | Date: July 31, 2026

Sprinklr

Google

1. Problem Overview

- n different recipes → 1D vector
- can be created → `recipes[i]` can be created if all `ingredient[i]` are present
- multiple ingredients may be there for `recipes[i]` → 2D vector
- supplies are given → 1D vector

2. Example Breakdown

Eg 1:

- `recipes = ["bread", "sandwich"]`
- bread needs: yeast, flour
- sandwich needs: bread, meat
- `supplies = ["yeast", "flour", "meat"]`

make bread and then we have
[bread, meat] → make sandwich

Hence a recipe becomes a new supply, no more.

Initially it might look an unordered map question, but it's a DAG (Directed Acyclic Graph).

3. Approach & Graph Concept

here indegree means → how many ingredients needed to make it

- In the Queue we push every available supply, reduce indegree while popping
- if `indegree == 0`, that recipe can now be used as a supply

GRAPH Construction

- for every recipe → `recipe[i]`
- and for every ingredient → `recipe[i][j]` (implied `ingredients[i][j]`)
- Create an edge: **ingredient** → **recipe**
- and `indegree[recipe]++`

QUEUE INITIALIZATION

- push every supply to Q, as they are not dependent on anyone

PROCESS using KAHN

- Implement Kahn's algorithm using the constructed graph and initialized queue.

4. C++ Code Implementation

```
class Solution {
public:
    vector<string> findAllRecipes(vector<string>& recipes, vector<vector<string>>& ingredients,
                                vector<string>& supplies) {
        //since recipe khud supply ban skta he according to eg:2
        //to dependency ka khel he
        //hence kahn algorithm
        //ingredient->recipe
        //ek baar indegree 0 hui to supply queue me push
        //initially queue me sirf supply waali cheeze

        //strings involve he to adjacency list ke jagah unordered map use kr rha

        unordered_map<string, vector<string>> adj; //ingredients->recipe
        unordered_map<string, int> indeg; //har recipe kitne pe dependant

        int n = recipes.size();
```

```

for(int i = 0; i < n; i++) {
    indeg[recipes[i]] = ingredients[i].size();
    for(string s : ingredients[i]) {
        adj[s].push_back(recipes[i]);
    }
}

queue<string> q;
//initially saare supplies q me jayege
for(string s : supplies) {
    q.push(s);
}

vector<string> ans;
while(!q.empty()) {
    string currs = q.front();
    q.pop();
    //ans update nhi krenenge kyunki pehla batch to initial ingredients the
    //jab bhi kisi ka indeg zero hoga tabhi ans update krenge
    //indeg 0 hona matlab ban ke taiyar ho gya

    for(auto &it : adj[currs]) {
        indeg[it]--;
        if(indeg[it] == 0) {
            q.push(it);
            ans.push_back(it);
        }
    }
}

return ans;
}
};

```